

Machine Learning Project 2025-2026

Multi-Agent Learning in Canonical Games
and Knights Archers Zombies

Fernando Alonzi r1084314

Zahra Lahrach r1087548

KU Leuven - Department of Computer Science
Supervisors: Giuseppe Marra, Wannes Meert

May 2026

Abstract

This report presents our work on multi-agent reinforcement learning (MARL) for the KU Leuven Machine Learning Project 2025-2026.

Task 2 analyzes independent multi-agent Q-learning in four matrix games, comparing ϵ -greedy, Boltzmann, and Lenient Boltzmann Q-learning using both empirical results and replicator dynamics.

Tasks 3a and 3b address the Knights-Archers-Zombies (KAZ) environment: training deep RL agents to control two archers and to detect zombies under distortions.

Code is available at: `aalst.student.cs.kuleuven.be:/cw/lvs/NoCsBack/vakken/HOT25A/ml-project/students/r1084314/`.

Contents

1	Task 1: Literature Study	3
2	Task 2: Learning & Dynamics in Matrix Games	3
2.1	Task 2.1: Nash equilibria and Pareto-optimal states	3
2.1.1	Stag Hunt	3
2.1.2	Subsidy Game.	4
2.1.3	Prisoner’s Dilemma.	4
2.1.4	Biased Rock-Paper-Scissors (RPS)	4
2.2	Task 2.2: Learning algorithms and experimental results	5
2.2.1	Algorithms	5
2.2.2	Experimental protocol	6
2.2.3	Stag Hunt	6
2.2.4	Subsidy Game	6
2.2.5	Prisoner’s Dilemma	7
2.2.6	Biased Rock-Paper-Scissors	8
2.2.7	Findings	8
2.3	Task 2.3: Replicator dynamics: analytical verification	9
2.4	Discussion	9
3	Task 3a: Knights-Archers-Zombies: RL Agent	9
3.1	Approach Overview	9
3.2	Research Questions and Design Choices	10
3.3	Network Architecture	10
3.4	Training Configuration	11
3.5	Evaluation	11
4	Task 3b: Knights-Archers-Zombies: Zombie Detection	11
4.1	Architecture	11
4.2	Dataset Collection and a Critical Bug Fix	11
4.3	Training	12
4.4	Results	12

1 Task 1: Literature Study

Three papers most directly influenced our approach.

Schulman et al. (2017) - Proximal Policy Optimization [13]. PPO is the backbone of our training pipeline. Its core idea is to limit how much the policy changes in each update, preventing unstable training while still allowing multiple gradient steps per batch of experience. This makes it simple to implement, stable to train, and well-suited to discrete action spaces. We chose PPO over value-based alternatives such as DQN because it naturally supports shared-parameter multi-agent training.

Yu et al. (2022) - MAPPO [15]. This work showed that PPO with parameter sharing achieves competitive or superior performance compared to specialised off-policy MARL algorithms on cooperative benchmarks, with minimal hyperparameter tuning. This directly justified our choice to train a single shared policy for both archers rather than two independent networks.

Engstrom et al. (2020) - Implementation Matters in Deep RL [14]. This paper showed that code-level optimisations; augmentations found only in implementations or described as details, are responsible for most of PPO’s empirical gains over TRPO. Applied to our context: our most impactful improvement was not an algorithmic change but an implementation one. Replacing the high-overhead environment wrapper chain with a direct environment call eliminated unnecessary CPU work, enabling 500 converged training iterations on CPU after over 30 GPU-hours yielded fewer than 100 iterations with the original setup.

2 Task 2: Learning & Dynamics in Matrix Games

We study independent multi-agent reinforcement learning in four benchmark matrix games: Stag Hunt, Subsidy Game, Prisoner’s Dilemma, and Biased Rock-Paper-Scissors (Biased RPS).

2.1 Task 2.1: Nash equilibria and Pareto-optimal states

A **Nash equilibrium** is a strategy profile where no agent would want to change its strategy if it knew what strategies the other agents were following.

Equilibria can be **pure** (deterministic: one action is always played with certainty) or **mixed** (stochastic: actions are played according to a probability distribution over the action set) [8].

A joint outcome (i, j) is **Pareto-optimal** if no other outcome makes every player at least as well off and at least one player strictly better off. [8].

2.1.1 Stag Hunt

	Stag (S)	Hare (H)
S	1, 1	0, $\frac{2}{3}$
H	$\frac{2}{3}$, 0	$\frac{2}{3}$, $\frac{2}{3}$

Nash equilibria : The Stag Hunt admits both.

- **Pure NEs:** each player commits to a single action with certainty:
 - **(S,S):** if the opponent always plays Stag, the best response is also Stag ($1 > \frac{2}{3}$). Both players are locked in.
 - **(H,H):** if the opponent always plays Hare, the best response is Hare ($\frac{2}{3} > 0$). Again, no one wants to deviate.
- **Mixed NE:**
- **Mixed NE:** instead of always choosing the same action, players randomise such that both actions yield the same expected payoff, Let y_S be the probability that the column player plays Stag. The row player’s expected payoffs are:

$$U(S) = y_S \cdot 1 + (1 - y_S) \cdot 0 = y_S, \quad U(H) = y_S \cdot 0 + (1 - y_S) \cdot \frac{2}{3} = \frac{2}{3}(1 - y_S).$$

Equating both expected payoffs by setting $U(S) = U(H)$ and solving:

$$y_S = \frac{2}{3}(1 - y_S) \Rightarrow \frac{5}{3}y_S = \frac{2}{3} \Rightarrow y_S^* = \frac{2}{5}.$$

By symmetry $x_S^* = \frac{2}{5}$ as well. The mixed NE is $(P(S) = \frac{2}{5}, P(H) = \frac{3}{5})$ for both players.

Pareto-optimal : Only (S,S), as it yields (1, 1) and Pareto dominates (H,H), which yields $(\frac{2}{3}, \frac{2}{3})$. No other outcome can improve one player without decreasing the payoff of the other.

2.1.2 Subsidy Game.

	S_1	S_2
S_1	12, 12	0, 11
S_2	11, 0	10, 10

Nash equilibria.

- **Pure NEs:** (S_1, S_1) at (12, 12) and (S_2, S_2) at (10, 10).
- **Mixed NE:** players randomise between S_1 and S_2 according to a probability distribution such that both actions yield the same expected payoff. Let y_{S_1} the probability that the column player chooses S_1 . The row player is indifferent when

$$12y_{S_1} = 11y_{S_1} + 10(1 - y_{S_1}),$$

which yields

$$y_{S_1}^* = \frac{10}{11} \approx 0.909.$$

Thus, each player chooses S_1 about 91% of the time and S_2 about 9% of the time.

Pareto-optimal. Only (S_1, S_1) is Pareto optimal, as it yields (12, 12) and strictly improves both players' payoffs compared to all other outcomes.

Note that (S_2, S_2) , although Pareto-dominated by (S_1, S_1) , is risk-dominant [8]; S_2 yields higher expected payoff under an uncertain opponent, making standard learners more likely to converge there.

2.1.3 Prisoner's Dilemma.

	Cooperate (C)	Defect (D)
C	-1, -1	-4, 0
D	0, -4	-3, -3

Nash equilibrium. Defect strictly dominates Cooperate ($0 > -1$ against C; $-3 > -4$ against D). Unique NE: (D,D) at $(-3, -3)$.

Pareto-optimal. (C, C) , (C, D) , and (D, C) are all Pareto-optimal [8]. Only (D, D) at $(-3, -3)$ is Pareto-dominated, since moving to (C, C) yields $(-1, -1)$, making both players strictly better off without making anyone worse off [8].

2.1.4 Biased Rock-Paper-Scissors (RPS)

$$A = \begin{pmatrix} 0 & -0.05 & 0.25 \\ 0.05 & 0 & -0.5 \\ -0.25 & 0.5 & 0 \end{pmatrix}, \quad B = -A \text{ (zero-sum)}.$$

Game structure. This is a zero-sum game, cyclic (R beats S, S beats P, P beats R) game, but the payoffs are asymmetric.

Why there is no pure Nash equilibrium. There is no pure strategy profile where both players are stable. Any fixed action can be exploited. Therefore, every action can be improved by switching, so no pure Nash equilibrium exists.

Mixed Nash equilibrium. Players randomise so that no action is strictly better than the others in expectation. This means all three actions must yield the same expected payoff under the opponent's strategy (y_R, y_P, y_S) :

$$U(R) = -0.05 y_P + 0.25 y_S, \quad U(P) = 0.05 y_R - 0.50 y_S, \quad U(S) = -0.25 y_R + 0.50 y_P.$$

Setting $U(R) = U(P)$ and $U(P) = U(S)$, and adding the simplex constraint $y_R + y_P + y_S = 1$, gives the system:

$$0.05 y_R + 0.05 y_P - 0.75 y_S = 0, \quad 0.30 y_R - 0.50 y_P - 0.50 y_S = 0, \quad y_R + y_P + y_S = 1.$$

Solving yields:

$$y_R^* = \frac{10}{16} = 0.625, \quad y_P^* = \frac{5}{16} = 0.3125, \quad y_S^* = \frac{1}{16} = 0.0625.$$

By symmetry $x^* = y^*$, so both players use the same distribution.

Pareto-optimality. In zero-sum games, improving one player's payoff necessarily decreases the other player's payoff, so Pareto improvements are not meaningful in this setting.

2.2 Task 2.2: Learning algorithms and experimental results

2.2.1 Algorithms

All three agents are *independent learners*: each applies a single-agent RL algorithm while treating the other player as part of the environment [1]. In these stateless matrix games ($\gamma = 0$, no next state), each player maintains a separate Q-table with one entry per action, and the Q-learning update [12] simplifies to:

$$Q(a) \leftarrow Q(a) + \alpha(r - Q(a)), \quad \alpha = 0.05. \quad (1)$$

The main challenge is the *exploration vs exploitation dilemma*: at each step, an agent must either exploit the current best-known action or explore other actions that are locally unoptimal but may yield better long-term rewards:

(a) **ε -greedy Q-learning** [9]. With probability $\varepsilon=0.1$ a random action is chosen uniformly (blind exploration); with probability $1-\varepsilon$ the greedy action $\arg \max_a Q(a)$ is chosen (exploitation: action with the highest Q-value). Exploration is therefore *blind*, all actions are explored equally, regardless of how good or bad their current Q-values appear.

(b) **Boltzmann Q-learning** [9, 10]. Actions a are sampled via a softmax distribution:

$$\pi(a) = \frac{\exp(Q(a)/\tau)}{\sum_b \exp(Q(b)/\tau)}, \quad \tau = 0.5. \quad (2)$$

Here τ (temperature) controls randomness. High τ leads to exploration; low τ makes the policy greedy [1] (better actions are sampled more often, and worse one have a smaller probability)

(c) **Lenient Boltzmann Q-learning** [2, 6]. Action selection is as in (b), but this variant addresses a fundamental problem in MARL: *relative overgeneralization*; a failure where optimal actions appear suboptimal because they are frequently penalized by a partner's early random exploration. To mitigate this, leniency buffers the last κ rewards per action and updates using only the **maximum** value. In effect, bad outcomes caused by a still-exploring partner are ignored, while good coordination outcomes always raise $Q(a)$:

$$Q(a) \leftarrow Q(a) + \alpha \left(\max_{k=1}^{\kappa} r_k(a) - Q(a) \right). \quad (3)$$

$Q(S)$ can now build momentum every time coordination succeeds, until $\pi(S)$ dominates and agents converge to the Pareto-Optimal equilibrium [6].

2.2.2 Experimental protocol

Evaluation consists of 12 independent 30,000-round runs per algorithm/game. Agents initialize with random policies and share no internal data. To smooth stochastic noise, we plot cumulative average policies recorded every 50 rounds [1].

Hyperparameters are fixed: $\alpha = 0.05, \varepsilon = 0.1, \tau = 0.5$; Lenient Boltzmann uses $\kappa = 5$ with decreasing temperature $\tau \leftarrow \max(0.999\tau, 0.1)$.

In figures, **circles** ($\bullet$) denote starts, **stars** ($\star$) final states, **crosses** ($+$) Nash equilibria, and **arrows** the replicator-dynamics vector field.

2.2.3 Stag Hunt

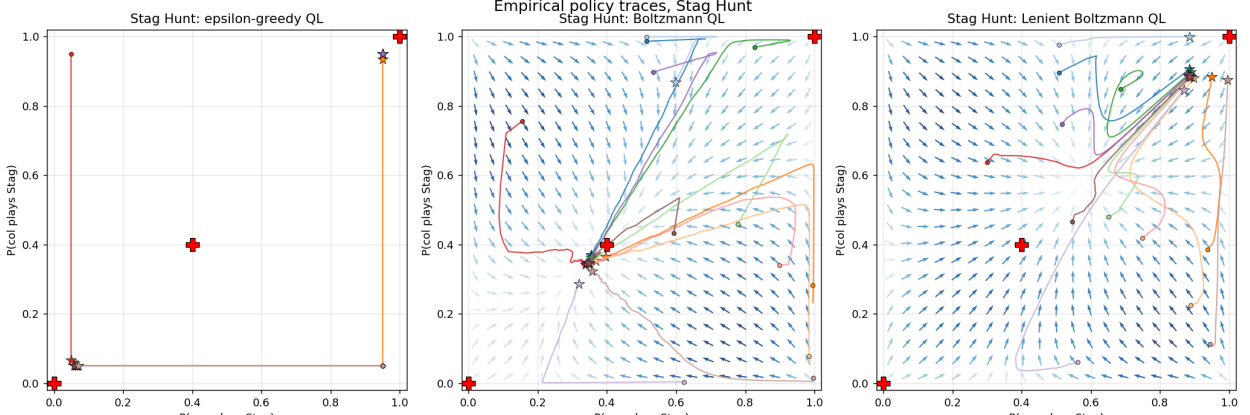


Figure 1: Stag Hunt: 12 trajectories per algorithm. Red $+$ marks Nash equilibria at $(0,0)$, $(1,1)$, and $(\frac{2}{5}, \frac{2}{5})$.

ε -greedy (left): every trajectory snaps immediately to one of the two pure NEs in the corners. Once $Q(S)$ or $Q(H)$ leads by chance, arg max locks in that action permanently and the small $\varepsilon = 0.1$ random exploration is too weak to escape (only 5% because it gets split between the 2 actions, which makes it even more difficult). About half the trajectories end at the suboptimal (H,H) .

Boltzmann (middle): Rather than converging to a pure NE, all trajectories cluster near the mixed NE saddle point at $(\frac{2}{5}, \frac{2}{5})$ without fully committing to either corner. This happens because at fixed temperature $\tau = 0.5$, the softmax function assigns strictly positive probability to every action: no action’s probability can ever reach exactly 0 or 1. This acts as a constant entropy force, continuously pulling agents away from pure strategies. As the vector field illustrates, Boltzmann’s continuous exploration prevents the agents from locking into the optimal (S,S) . Instead, they remain trapped near the saddle point, earning expected rewards well below the Pareto-optimal payoff of $(1,1)$ [1, 10].

Lenient Boltzmann (right): most trajectories reach the Pareto-optimal (S,S) corner. The lenient update (Eq. 3) stores the last κ rewards per action and updates Q using only the maximum. Intuitively, the probability that at least one of κ interactions yields a successful coordination reward is $1 - (1 - y_S)^\kappa$, which grows quickly with κ even when y_S is low. With $\kappa = 5$, $Q(S)$ remains high despite frequent miscoordinations, protecting the agent from being punished for a still-exploring partner [6]. Some trajectories deviate from the vector field because the field is computed at fixed $\tau = 0.5$ while the actual training uses a decaying temperature ($\tau_0 = 1.0$ down to 0.1), causing the true dynamics to shift throughout training. Nevertheless, convergence to (S,S) is achieved in the majority of runs, confirming the *relative overgeneralization* correction predicted by Panait et al. [6].

2.2.4 Subsidy Game

ε -greedy (left): Trajectories converge to the sub-optimal NE. None of them converge to the Pareto-Optimal (S_1, S_1) . The 10% random exploration is too weak to escape once $Q(S_2)$ leads.

Boltzmann (middle): The algorithm struggles significantly to coordinate on the Pareto-optimal (S_1, S_1) outcome. As the vector field reveals, convergence to the risk-dominant (S_2, S_2) equilibrium dominates almost the entire state space. The mixed Nash equilibrium at roughly $(0.91, 0.91)$ acts as

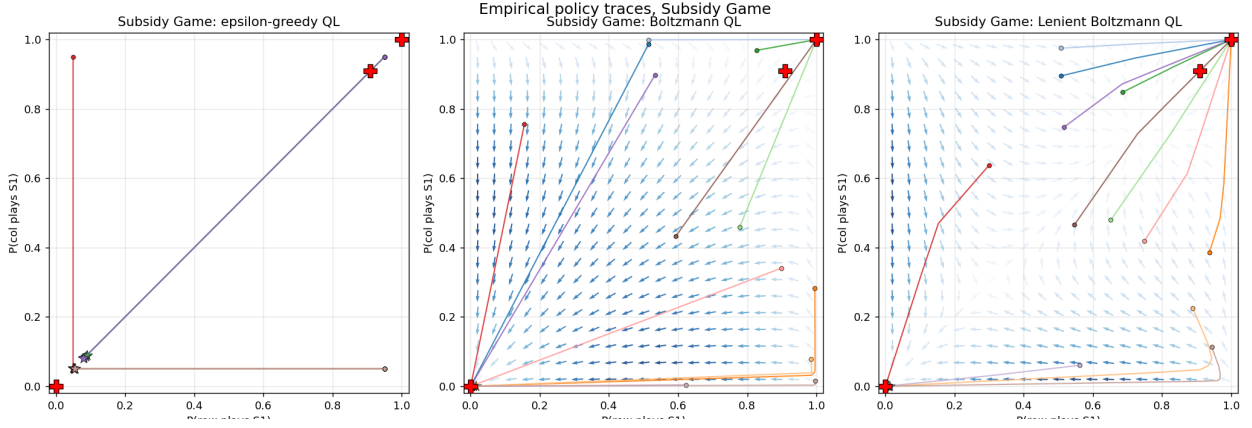


Figure 2: Subsidy Game: 12 trajectories per algorithm. Red + marks Nash equilibria at $(0,0)$, $(1,1)$, and $(\frac{10}{11}, \frac{1}{11})$.

a severe tipping point. Only trajectories initialized close to the top-right corner have any chance of converging to (S_1, S_1) . Everywhere else, the asymmetric payoffs create a gradient towards (S_2, S_2) . If an agent attempts S_1 while their partner explores S_2 , they receive 0, whereas playing S_2 guarantees a safe minimum reward of 10. This safety net inflates $Q(S_2)$ so heavily during early exploration that it permanently traps the vast majority of trajectories at the suboptimal equilibrium [1].

Lenient Boltzmann (right): leniency improves convergence to the Pareto-optimal (S_1, S_1) . For most of the state space, the κ -buffer successfully protects agents from early miscoordination penalties, guiding trajectories toward the optimal outcome. However, trajectories originating along the extreme left or bottom margins fail to build momentum and fall into the risk-dominant (S_2, S_2) NE. High initial temperature ($\tau_0 = 1.0$) compounds this, early stochasticity can push trajectories from favourable starting positions into these marginal regions before they stabilise. The vector field explains this behaviour, revealing an L-shaped dividing boundary along the axes. The κ -buffer shifts the gradient toward (S_1, S_1) across the vast majority of the state space. However, if *either* player starts with an extreme bias toward S_2 , their partner plays S_1 so rarely that even a $\kappa = 5$ buffer is unlikely to capture the optimal payoff. Overall, leniency effectively mitigates relative overgeneralization for the vast majority of starting conditions, provided early exploration does not push agents into these marginal traps [6].

2.2.5 Prisoner's Dilemma

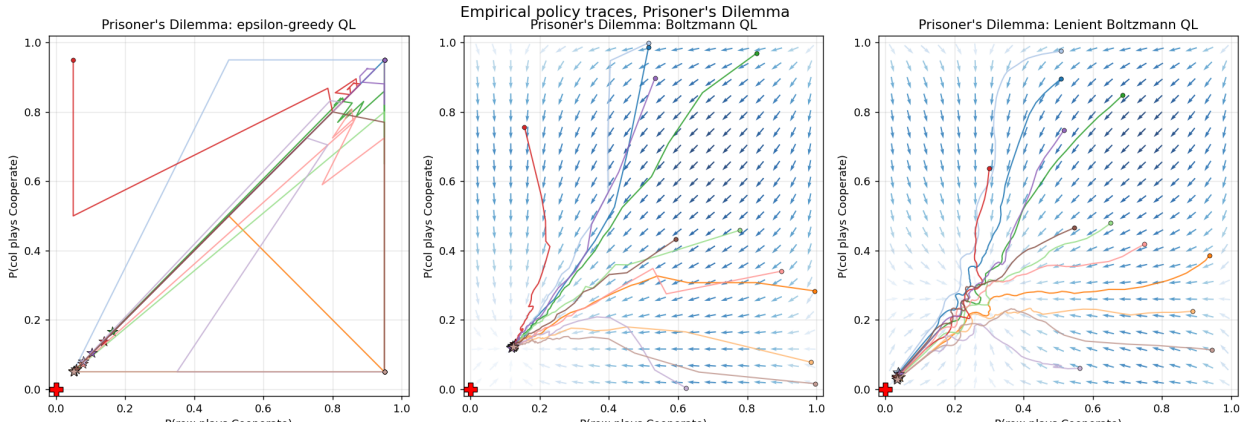


Figure 3: Prisoner's Dilemma: 12 trajectories per algorithm. Red + marks the unique NE at $(0,0) = (D,D)$.

Independent RL agents only maximize their own rewards, so they cannot reach the global Pareto-optimal (C,C) outcome. Because Defect strictly dominates Cooperate, $Q(D)$ always grows faster than $Q(C)$. As a result, the vector fields point entirely toward the mutual Defect (D,D) Nash equilibrium. However, because of exploration mechanics, no algorithm perfectly reaches the exact $(0,0)$ corner.

ε -greedy (left) stabilises exactly at $P(C) = 0.05$: with $\varepsilon = 0.1$ and two actions, Cooperate is played with probability $\varepsilon/2 = 0.05$ at all times, permanently preventing full commitment to Defect. Standard Boltzmann (middle) stalls near $(0.1, 0.1)$ because its fixed temperature ($\tau = 0.5$) forces continuous exploration [10]. Lenient Boltzmann (right) also stops short because its temperature only drops to a minimum floor of $\tau = 0.1$. In fact, leniency makes the learning paths more erratic and slows down convergence. The κ -buffer remembers the rare “best” payoffs for Cooperate, making a bad action look deceptively safe. This confirms that leniency offers no advantage in strictly dominant games and actually hurts convergence speed [6].

2.2.6 Biased Rock-Paper-Scissors

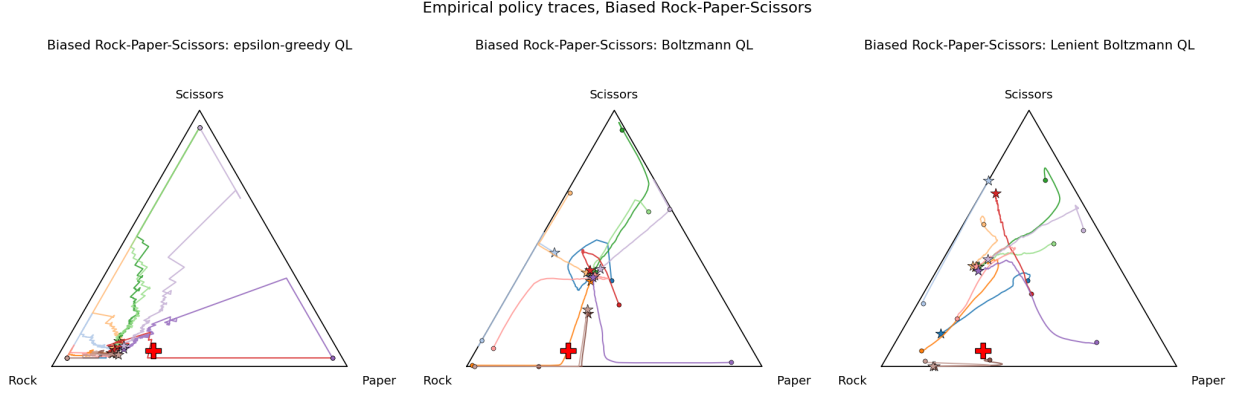


Figure 4: Biased RPS (row player’s strategy on the simplex). Red + marks the unique mixed NE at $(\frac{10}{16}, \frac{5}{16}, \frac{1}{16})$.

Biased Rock-Paper-Scissors: The algorithms show distinct behaviours, and none perfectly stabilise at the mixed Nash equilibrium.

ε -greedy (left) completely misses the game’s cyclic nature, with trajectories collapsing into the Rock corner. Because Rock has the highest equilibrium probability ($\frac{10}{16}$), it often secures the highest early Q-value, causing the greedy policy to permanently lock onto it.

Standard Boltzmann (middle) successfully captures the game’s cyclic nature, with trajectories spiraling inward and clustering near the mixed equilibrium. This spiraling occurs because zero-sum games are essentially endless loops of counters and counter-attacks; the algorithms continuously chase each other in a cycle rather than stopping at a fixed point [7, 10].

Lenient Boltzmann (right) also spirals, but with much wider, more erratic paths. Since this is a strictly competitive game, the agents are opponents, meaning there is no cooperative teamwork to protect. The lenient κ -buffer simply causes agents to hold onto old, lucky payouts, making them stubbornly repeat outdated moves even after the opponent has adapted. This delayed reaction destabilises the learning process and scatters the trajectories further from the center, completely backfiring compared to its success in cooperative games. [6].

2.2.7 Findings

3 findings emerge:

(1) **Coordination games.** Standard Boltzmann suffers from *relative overgeneralization* [6]: early miscoordination makes the globally optimal action look unprofitable. Lenient Boltzmann filters those low rewards and drastically improves convergence to the Pareto-optimal equilibrium in both the Stag Hunt and Subsidy games.

(2) **Social dilemma.** In the Prisoner’s Dilemma, all algorithms converge toward (D,D). Leniency provides no advantage when one action strictly dominates; no level of optimism can overcome the fact that Defect yields a higher payoff regardless of the partner’s strategy.

(3) **Zero-sum games.** Biased RPS produces sustained cycling. This aligns with theoretical predictions for strictly competitive games: because agents are locked in an endless loop of optimal counter-attacks, the dynamics produce continuous cycles rather than stabilising at a fixed point (Hamiltonian structure) [7].

2.3 Task 2.3: Replicator dynamics: analytical verification

Beyond running experiments, we can mathematically predict how the learning algorithms should behave using *replicator dynamics*, a system of differential equations from evolutionary game theory that describes how strategies evolve over time. Tuyls et al. [10] showed that Boltzmann Q-learning follows a *selection-mutation* model (Eqs. 7-8; restated as Eq. 10 in Bloembergen et al. [1]):

$$\dot{x}_i = \underbrace{\frac{\alpha}{\tau} x_i [(Ay)_i - x^\top Ay]}_{\text{selection: above-average actions grow}} - \underbrace{\alpha x_i [\ln x_i - \sum_k x_k \ln x_k]}_{\text{mutation: entropy prevents pure strategies}}. \quad (4)$$

The selection term rewards actions that perform above average; the mutation term is the mathematical consequence of Boltzmann exploration, which permanently prevents any action’s probability from reaching exactly 0 or 1. For Lenient Boltzmann, the expected payoff $(Ay)_i$ is replaced by the lenient utility u_i (Bloembergen et al. [1], Eq. 11):

$$u_i = \sum_j a_{ij} y_j \frac{\left(\sum_{k: a_{ik} \leq a_{ij}} y_k \right)^\kappa - \left(\sum_{k: a_{ik} < a_{ij}} y_k \right)^\kappa}{\sum_{k: a_{ik} = a_{ij}} y_k}, \quad (5)$$

which is the expected *maximum* payoff over κ draws; mathematically encoding the leniency mechanism directly into the dynamics.

We computed both vector fields on a 21×21 grid and overlaid them on the empirical trajectories (middle and right panels of Figures 1-3), allowing direct visual comparison between theory and experiment. Three observations confirm the theory: in the Stag Hunt, the mixed NE acts as a saddle point under Boltzmann dynamics, with leniency expanding the convergence to the Pareto Optimal state (S,S); in the Subsidy Game, the Boltzmann field strongly favours the risk-dominant NE (S_2, S_2) while the Lenient field redirects flow toward the Pareto-optimal (S_1, S_1) ; in the Prisoner’s Dilemma, both fields point toward (D,D) since leniency cannot alter a strict dominance relationship.

The empirical traces generally align with the vector field streamlines, though stochastic noise causes deviations; particularly during high-temperature exploration. The replicator model accurately captures global convergence trends but remains a deterministic approximation that cannot predict individual trajectory variance [1].

2.4 Discussion

The central finding of this task is that **leniency matters far more than the choice of exploration mechanism**. Lenient Boltzmann consistently reaches the Pareto-optimal NE in coordination games, while ε -greedy and standard Boltzmann fail. This confirms the claim of Panait et al. [6]: “the performance of these algorithms depends primarily on the level of lenience that the agents exhibit to one another.” The replicator-dynamics analysis explains why: replacing the expected payoff with the expected maximum payoff (Eq. 5) reshapes the convergence landscape, steering agents away from risk-dominant suboptimal equilibria [6].

3 Task 3a: Knights-Archers-Zombies: RL Agent

3.1 Approach Overview

We implemented a 2 stage hybrid pipeline separating perception from decision-making. During training, a lightweight MLP policy is trained via Proximal Policy Optimisation (PPO) [13] on a

compact 48-dimensional feature vector built directly from privileged game state. At submission time, the same vector is reconstructed from CNN-detected zombie bounding boxes (from pixels, as required) and archer positions read from `env.agent_list` (permitted by the instructors). This separation means the RL training loop never touches pixels, making training faster and the gradient signal cleaner.

Key engineering fix: Our initial training pipeline routed observations through the full `create_environment` wrapper chain, which rendered distortion effects even at `distortion_level=0` and imposed heavy per-step CPU overhead. Replacing this with a direct call to `knights_archers_zombies_v10.env` followed by SuperSuit’s `black_death_v3` reduced environment step time dramatically, consistent with [14]’s finding that implementation details dominate PPO performance. GPU utilisation was negligible because environment simulation, not gradient computation, was the bottleneck. This reduced per-step CPU overhead substantially, enabling 500 training iterations on CPU in a feasible time budget.

3.2 Research Questions and Design Choices

RQ1: Should the policy train on pixels or on a compact feature vector?

The 48-dimensional observation vector is structured as:

$$\mathbf{o} = \underbrace{[x_s, y_s, d_s^x, d_s^y]}_{\text{self}}, \underbrace{[x_t, y_t, d_t^x, d_t^y]}_{\text{teammate}}, \underbrace{[x_1, y_1, w_1, h_1, p_1, \dots, x_8, y_8, w_8, h_8, p_8]}_{\text{8 zombie slots}} \in \mathbb{R}^{48} \quad (6)$$

Positions are normalised by screen dimensions (1280×720), directions are unit vectors, $p_i \in \{0, 1\}$ is a presence flag, and all values are clipped to $[-2, 2]$. Zombies are sorted by descending y (most threatening first). Including teammate state enables each archer to condition on the other’s behaviour under parameter sharing, supporting emergent coordination [15]. Bounding-box dimensions w_i, h_i are included to match the submission-time CNN output format, keeping the training and inference vectors structurally identical.

RQ2: Should we use reward shaping?

We experimented extensively with reward shaping. An initial wrapper amplified kills by $\times 10$ and deaths by $\times 5$, and added per-step aim rewards ($+0.01 \cdot \max(0, \cos \theta)$) and an attack bonus ($+0.05$ when firing with $\cos \theta > 0.7$). Pilot runs revealed that this dense shaping produced reward hacking: agents learned to face zombies continuously without shooting, accumulating up to ~ 6 reward per episode purely from aim tracking while achieving very few kills. This is a well-documented failure mode of shaped rewards [17]. We ultimately removed all shaping and trained on the base PettingZoo reward ($+1$ per kill, -1 per death), which aligns the training objective precisely with the evaluation metric. The sparse signal was sufficient given the large 10 000-step batch size.

RQ3: How should multi-agent training be structured?

We used a fully parameter-shared policy: both archers share identical network weights, with each archer’s own state appearing in the first four dimensions of its observation. Parameter sharing halves the effective sample cost per gradient update and has been shown to outperform independent training for homogeneous cooperative agents [15, 16].

3.3 Network Architecture

The policy is a small actor-critic MLP:

$$\mathbf{h} = \text{ReLU}(\mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{o} + \mathbf{b}_1) + \mathbf{b}_2), \quad \mathbf{W}_1 \in \mathbb{R}^{128 \times 48}, \mathbf{W}_2 \in \mathbb{R}^{128 \times 128}$$

The shared trunk $\mathbf{h} \in \mathbb{R}^{128}$ feeds a policy head (128→6 action logits) and a value head (128→1). Action selection uses stochastic sampling from $\text{Categorical}(\text{softmax}(\mathbf{h}_\pi))$, preserving exploration at inference time.

3.4 Training Configuration

Training used PPO via RLlib [19] with the legacy stack (`enable_rl_module_and_learner=False`) for compatibility.

Key hyperparameters: `train_batch_size=10 000`, `minibatch_size=1 000`, `num_epochs=4`, `lr=3 × 10-4`, `entropy_coeff=0.03`, `grad_clip=0.5`, `max_zombies=4`, `max_cycles=2 500`, `distortion_level=0`.

The large batch size (10 000 steps) was chosen to ensure that each policy update contained statistically meaningful kill events given sparse rewards [13]. The entropy coefficient of 0.03 is deliberately elevated above the RLlib default (0.0) to sustain exploration during the early sparse-reward phase. Training ran for approximately 500 iterations on CPU, reaching `episode_return_mean` ≈ 65 at iteration 500.

3.5 Evaluation

Table 1 compares our agent to a random baseline across distortion levels (5 episodes, seed 42, 240s episode time limit). Our agent outperforms random at every distortion level. At distortion 0 the gain is $6.65\times$ (26.60 vs. 4.00); at distortions 3 and 5 the gains are $1.70\times$ and $2.63\times$ respectively, with episode lengths roughly 30% longer, indicating the agent survives longer even under corrupted observations.

We note that local evaluation results show high variance across runs: scores at distortion 0 ranged from approximately 4 to 100 depending on the random seed.

The final tournament score was **42.4**, confirming generalisation beyond fixed local seeds. Performance degrades at higher distortions because CNN detection noise propagates into the input vector, a known limitation of decoupled perception.

Table 1: Evaluation results (5 episodes, seed 42). T=terminated, TO=timeout.

Agent	Distortion	Avg Reward	Avg Ep. Length	Endings
Ours	0	26.60	1303.4	5T
Ours	3	7.80	453.0	2T, 3TO
Ours	5	10.00	493.8	3T, 2TO
Random	0	4.00	401.0	5T
Random	3	4.60	421.8	4T, 1TO
Random	5	3.80	377.0	5T

4 Task 3b: Knights-Archers-Zombies: Zombie Detection

4.1 Architecture

We trained a custom convolutional neural network (**ZombieCNN**) for single-class zombie detection. The input is a pixel observation resized to $3 \times 90 \times 160$. The backbone applies 3 strided convolutions ($3 \rightarrow 16 \rightarrow 32 \rightarrow 32$ channels, stride 2 each, BatchNorm + ReLU), yielding an $8 \times$ spatial downsampling to a 12×20 feature map. A YOLO-style [18] detection head (Conv $32 \rightarrow 64$, Conv $64 \rightarrow 5$) produces one prediction per grid cell: (conf, δx , δy , w , h). Absolute box centres are recovered as:

$$x_{\text{abs}} = \frac{g_x + \sigma(\delta x)}{G_w}, \quad y_{\text{abs}} = \frac{g_y + \sigma(\delta y)}{G_h}$$

where (g_x, g_y) is the cell index and $G_w = 20$, $G_h = 12$. Width and height are predicted directly in normalised image coordinates via sigmoid. Post-processing applies a confidence threshold (0.7 for policy inference, 0.6 for the detection submission) followed by greedy Non-Maximum Suppression (IoU threshold 0.4) to remove duplicate detections from adjacent cells.

4.2 Dataset Collection and a Critical Bug Fix

The training dataset was collected by running the KAZ environment with random actions and recording frame; An initial dataset of 32 968 frames was collected across 200 episodes with random distortion levels (0-5), saving frames at 320×180 resolution.

A critical bug was found in the original collection script: using action 0 (move forward) caused archers to walk into zombie clusters, die in the first 20-30 steps, and produce a dataset where zombies appeared only at the top 20% of the screen.

The CNN trained on this data learned this spatial bias and incorrectly detected archers as zombies, producing up to ~ 17 false positives per frame. The fix was to sample actions uniformly from $\{1, 2, 3, 5\}$ (rotate and stay), keeping archers away from zombies and not attacking them in order to keep them as long as possible on the screen.

4.3 Training

The CNN was trained with AdamW ($\text{lr} = 5 \times 10^{-4}$, weight decay 10^{-4}) for 50 epochs with a cosine learning rate schedule and 3-epoch linear warmup. Gradient clipping (max norm 1.0) was applied. Data augmentation included random horizontal flips (with consistent label mirroring: $x \leftarrow 1 - x - w$) and colour jitter (brightness $\pm 15\%$, contrast shift $\pm 7.5\%$). The composite YOLO loss is:

$$\mathcal{L} = \lambda_{\text{obj}} \mathcal{L}_{\text{obj}} + \lambda_{\text{noobj}} \mathcal{L}_{\text{noobj}} + \lambda_{\text{bbox}} \mathcal{L}_{\text{bbox}}$$

with $\lambda_{\text{obj}} = 5$, $\lambda_{\text{noobj}} = 0.5$, $\lambda_{\text{bbox}} = 5$. $\mathcal{L}_{\text{obj}}$ and $\mathcal{L}_{\text{noobj}}$ are binary cross-entropy on confidence; $\mathcal{L}_{\text{bbox}}$ is Smooth-L1 on box coordinates for positive cells only. The 90/10 train/validation split was fixed with seed 42.

4.4 Results

The official evaluator returned an average precision of **1.0** on the provided test image (distortion level 0), confirming the model detects zombies precisely above the 75% threshold required for full marks. The gap between the validation P/R metrics and the 1.0 AP on the official evaluator is explained by the validation set containing random distortion levels 0-5, which is a harder distribution than the undistorted test image; the model was saved at best validation loss (epoch 31, val loss 0.1204), corresponding to the optimal generalisation point under this harder distribution.

Conclusion: A lightweight YOLO-style CNN trained on a carefully curated dataset achieves perfect detection precision at distortion 0. The dataset collection bug (action=0 causing archer; zombie spatial conflation) was the single most impactful fix for detection quality.

Time spent per person

Code for all tasks is stored at:

`aalst.student.cs.kuleuven.be:/cw/lvs/NoCsBack/vakken/H0T25A/ml-project/students/r1084314/`

Sub-task	Zahra	Fernando
Task 1: Literature study	6	4
Task 2.1: Analytical equilibria	3	2
Task 2.2: Implementation + plots	5	3
Task 2.3: Replicator dynamics	4	2
Task 3a: KAZ RL agent	25	13
Task 3b: Zombie detection	9	25
Report	17	5
Total	69	54

References

- [1] D. Bloembergen, K. Tuyls, D. Hennes, M. Kaisers. Evolutionary dynamics of multi-agent learning: a survey. *Journal of Artificial Intelligence Research*, 53:659-697, 2015. <https://www.jair.org/index.php/jair/article/download/10952/26090/20434>
- [2] D. Bloembergen, M. Kaisers, K. Tuyls. Lenient frequency adjusted Q-learning. *Proc. BNAIC*, pp. 19-26, 2010. <https://lirias.kuleuven.be/2261410>
- [3] D. Bloembergen, M. Kaisers, K. Tuyls. Empirical and theoretical support for lenient learning. *Proc. AAMAS*, pp. 1105-1106, 2011.
- [4] T. Börgers, R. Sarin. Learning through reinforcement and replicator dynamics. *Journal of Economic Theory*, 77(1):1-14, 1997.
- [5] L. Busoniu, R. Babuska, B. De Schutter. A comprehensive survey of multiagent reinforcement learning. *IEEE Trans. Systems, Man, and Cybernetics, Part C*, 38(2):156-172, 2008.
- [6] L. Panait, K. Tuyls, S. Luke. Theoretical advantages of lenient learners: an evolutionary game theoretic perspective. *Journal of Machine Learning Research*, 9:423-457, 2008. <https://www.jmlr.org/papers/volume9/panait08a/panait08a.pdf>
- [7] Y. Sato, J. P. Crutchfield. Coupled replicator equations for the dynamics of learning in multiagent systems. *Physical Review E*, 67:015206, 2002. <https://arxiv.org/pdf/nlin/0204057>
- [8] Y. Shoham, K. Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2009. <http://www.masfoundations.org/mas.pdf>
- [9] R. S. Sutton, A. G. Barto. *Reinforcement Learning: An Introduction* (2nd edition). MIT Press, 2018. <http://incompleteideas.net/book/the-book-2nd.html>
- [10] K. Tuyls, K. Verbeeck, T. Lenaerts. A selection-mutation model for Q-learning in multi-agent systems. *Proc. AAMAS*, pp. 693-700, 2003. <https://doi-org.kuleuven.e-bronnen.be/10.1145/860575.860687>
- [11] K. Tuyls, G. Weiss. Multiagent learning: basics, challenges, and prospects. *AI Magazine*, 33(3):41-52, 2012. https://www.researchgate.net/publication/255739006_Multiagent_Learning_Basics_Challenges_and_Prospets
- [12] C. J. C. H. Watkins, P. Dayan. Q-learning. *Machine Learning*, 8(3):279-292, 1992.
- [13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv:1707.06347*, 2017.
- [14] L. Engstrom, A. Ilyas, S. Santurkar, D. Tsipras, F. Janoos, L. Rudolph, and A. Madry. Implementation matters in deep RL: A case study on PPO and TRPO. In *ICLR*, 2020.
- [15] C. Yu, A. Velu, E. Vinitzky, J. Gao, Y. Wang, A. Bayen, and Y. Wu. The surprising effectiveness of PPO in cooperative multi-agent games. In *NeurIPS*, 2022.
- [16] F. Christianos, G. Papoudakis, M. A. Rahman, and S. V. Albrecht. Scaling multi-agent reinforcement learning with selective parameter sharing. In *ICML*, 2021.
- [17] A. Y. Ng, D. Harada, and S. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *ICML*, 1999.
- [18] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. In *CVPR*, 2016.
- [19] E. Liang, R. Liaw, R. Nishihara, P. Moritz, R. Fox, K. Goldberg, J. Gonzalez, M. Jordan, and I. Stoica. RLlib: Abstractions for distributed reinforcement learning. In *ICML*, 2018.